Production C++20 User-Space USB Audio Class Driver and HID DSP Subsystem on Android Usbfs

Standard Android audio architectures route audio streams through the client-side AudioTrack API into the native AudioFlinger service, which interacts with the hardware abstraction layer (HAL) before reaching the kernel-level ALSA driver. This processing pipeline introduces mandatory sample-rate conversion, floating-point to integer truncations, volume scaling, and software mixer buffering latencies. Furthermore, Android's framework layer completely lacks native facilities for bit-perfect Direct Stream Digital (DSD) bitstreaming and prevents direct interaction with vendor-specific digital signal processing (DSP) hardware embedded in modern USB Type-C In-Ear Monitors (IEMs).
Bypassing the entire framework stack is accomplished by obtaining the underlying raw Linux USB device file descriptor from the Android Open Source Project (AOSP) framework via android.hardware.usb.UsbDeviceConnection.getFileDescriptor(). When passed across the Java Native Interface (JNI) to a native C++20 shared library, this integer represents an open file descriptor to the virtual Linux USB device character node (/dev/bus/usb/XXX/YYY) handled by the kernel's usbfs driver (<linux/usbdevice_fs.h>). Implementing a production-grade, asynchronous USB Audio Class 1.0 (UAC1) and 2.0 (UAC2) user-space driver directly atop usbfs requires mastering four interconnected technical domains: zero-copy binary configuration descriptor parsing, microframe-synchronized asynchronous feedback calculation, multi-rate DSD/DoP framing, and USB Human Interface Device (HID) parametric equalizer (PEQ) control bridging.

Complete Descriptor Parsing Byte Layouts

Initializing communication with an audio device requires reading and deserializing the entire configuration tree. Unlike single-descriptor inquiries, the host reads the complete, contiguous configuration block using a single USBDEVFS_CONTROL ioctl on Endpoint 0 targeting the USB_DT_CONFIG descriptor type. This linear binary payload concatenates the Configuration Descriptor, standard Interface Descriptors, Class-Specific Audio Control (CS_INTERFACE) descriptors, Class-Specific Audio Streaming descriptors, and standard plus class-specific Endpoint Descriptors in an unbroken byte sequence.

Configuration Space Traversal and Topology Resolution

The parsing state machine must track hierarchical interface boundaries while reading the linear descriptor buffer. In multi-function USB audio devices, the audio interface collection is defined by an Interface Association Descriptor (IAD). The IAD signals that the grouped interfaces belong to the Audio Interface Function (bFunctionClass = 0x01), where bFunctionProtocol explicitly specifies whether the sub-interfaces operate under UAC1 (0x00) or UAC2 (0x20).
Within this association, the driver encounters two mandatory interface subtypes:

1. The **Audio Control (AC) Interface** (bInterfaceSubClass = 0x01): Exposes topological

entities including Input Terminals, Output Terminals, Feature Units (volume/mute), Mixer Units, and—under UAC2—Clock Source and Clock Selector entities.
2. The **Audio Streaming (AS) Interface** (bInterfaceSubClass = 0x02): Exposes alternate settings where Alternate Setting 0 is an operational zero-bandwidth profile (no active isochronous endpoints), and Alternate Settings 1 through N declare active data streaming and feedback endpoints, subslot sample bit depths, and format allocations.

Traversing these descriptors in C++20 relies on standard memory layout mapping via packed structures and pointer advancing. The offset increases by the bLength field located at byte offset 0 of each header. Descriptors with bLength == 0 or sizes that overflow the total configuration buffer indicate corrupted descriptors or malformed device firmware and must be rejected immediately to prevent heap and buffer over-reads.

Structural Layout Divergence: UAC1 vs. UAC2

The structural definitions governing UAC1 and UAC2 descriptors exhibit critical differences in entity categorization, clock discovery, and streaming formats.

Structural Parameter	USB Audio Class 1.0 (UAC1)	USB Audio Class 2.0 (UAC2)	Functional Impact on Driver Architecture
Interface Protocol	0x00 (Defined by SubClass)	0x20 (IP_VERSION_02_00)	Selects the active parsing engine and entity resolution rules.
AC Header Descriptor	Contains bInCollection and the baInter[span_17](start_span)[span_17](end_span)faceNr[] array.	Fixed 9-byte structure; interfaces are bound via IAD.	UAC1 binds streaming interfaces inside the AC header; UAC2 uses external descriptors.
Clock Representation	Implicitly derived from USB bus Start-of-Frame (SOF) markers.	Explicit CLOCK_SOURCE (0x0B) and CLOCK_SELECTOR entities.	UAC2 sample-rate negotiation requires querying clock entities rather than endpoints.
Input Terminal Descriptor	12 bytes; wChann[span_54](start_span)[span_54](end_span)elConfig is a 16-bit bitmask.	17 bytes; bmChannelConfig is a 32-bit spatial mask; contains bCSourceID.	UAC2 associates the input stream with a specific clock domain entity ID.
Feature Unit Controls	bmaControls array of 16-bit masks per audio channel.	bmaControls array of 32-bit masks supporting 4-state capability queries.	UAC2 supports separate read/write control capabilities for master and individual channels.
Format Type I Descriptor	Dynamic length; lists sample rates (tSamFreq) directly as 24-bit discrete integers.	Fixed 6 bytes; defines bSubslotSize and bBitResolution only.	UAC1 extracts valid sample rates from the format descriptor; UAC2 reads them via clock control requests.
Feedback Endpoint	Full-Speed:	High-Speed: Dedicated	Dictates explicit versus

Structural Parameter	USB Audio Class 1.0 (UAC1)	USB Audio Class 2.0 (UAC2)	Functional Impact on Driver Architecture
Type	Synchronous or Adaptive Interrupt/Isochronous IN.	Isochronous IN endpoint with usage type 0x01.	implicit feedback synchronization mechanics.

C++20 Descriptor Structures and Zero-Copy Parsing Engine

The binary parsing implementation below uses strict C++20 concepts, strong typing, and direct memory overlays mapped to packed structures matching the USB Device Class Definition for Audio Devices:

```
#pragma once

#include <cstdint>
#include <cstddef>
#include <span>
#include <vector>
#include <optional>
#include <bit>
#include <string>
#include <system_error>

#pragma pack(push, 1)

// Standard USB Descriptor Base Header
struct UsbDescriptorHeader {
    uint8_t  bLength;
    uint8_t  bDescriptorType;
};

// Interface Association Descriptor (IAD)
struct UsbIadDescriptor {
    uint8_t  bLength;           // 0x08
    uint8_t  bDescriptorType;    // 0x0B (DT_INTERFACE_ASSOCIATION)
    uint8_t  bFirstInterface;
    uint8_t  bInterfaceCount;
    uint8_t  bFunctionClass;     // 0x01 (AUDIO)
    uint8_t  bFunctionSubClass;  // 0x00
    uint8_t  bFunctionProtocol;  // 0x20 (AF_VERSION_02_00) or 0x00
    uint8_t  iFunction;
};

// Standard USB Interface Descriptor
struct UsbInterfaceDescriptor {
    uint8_t  bLength;           // 0x09
    uint8_t  bDescriptorType;    // 0x04 (DT_INTERFACE)
    uint8_t  bInterfaceNumber;
```

```
    uint8_t  bAlternateSetting;
    uint8_t  bNumEndpoints;
    uint8_t  bInterfaceClass;    // 0x01 (AUDIO)
    uint8_t  bInterfaceSubClass; // 0x01 (AUDIOCONTROL), 0x02 (AUDIOSTREAMING), 0x03
(MIDISTREAMING)
    uint8_t  bInterfaceProtocol; // 0x00 (UAC1), 0x20 (UAC2)
    uint8_t  iInterface;
};

// Standard USB Isochronous Audio Endpoint Descriptor
struct UsbAudioEndpointDescriptor {
    uint8_t  bLength;           // 0x07 or 0x09
    uint8_t  bDescriptorType;    // 0x05 (DT_ENDPOINT)
    uint8_t  bEndpointAddress;   // D7: 0=OUT, 1=IN; D3..0: Endpoint number
    uint8_t  bmAttributes;       // D1..0: 01=Isochronous; D3..2: Sync (01=Async, 10=Adaptive)
                                 // D5..4: Usage (00=Data, 01=Feedback, 10=Implicit Feedback)
    uint16_t wMaxPacketSize;
    uint8_t  bInterval;          // Polling interval (frames for FS, microframes for HS)
    uint8_t  bRefresh;           // Feedback refresh rate: 2^bRefresh microframes
    uint8_t  bSynchAddress;      // Synch endpoint address for explicit feedback
};

// ========================================
// UAC1 Specific Descriptors (CS_INTERFACE: 0x24)
// ========================================

struct Uac1HeaderDescriptor {
    uint8_t  bLength;
    uint8_t  bDescriptorType;       // 0x24 (CS_INTERFACE)
    uint8_t  bDescriptorSubtype;    // 0x01 (HEADER)
    uint16_t bcdADC;                // 0x0100
    uint16_t wTotalLength;
    uint8_t  bInCollection;
};

struct Uac1InputTerminalDescriptor {
    uint8_t  bLength;           // 0x0C
    uint8_t  bDescriptorType;    // 0x24
    uint8_t  bDescriptorSubtype; // 0x02 (INPUT_TERMINAL)
    uint8_t  bTerminalID;
    uint16_t wTerminalType;      // 0x0101: USB Streaming, 0x0201: Microphone
    uint8_t  bAssocTerminal;
    uint8_t  bNrChannels;
    uint16_t wChannelConfig;
    uint8_t  iChannelNames;
    uint8_t  iTerminal;
};
```

```
struct Uac1FormatTypeIDescriptor {
   uint8_t  bLength;           // 8 + (3 * n)
   uint8_t  bDescriptorType;    // 0x24
   uint8_t  bDescriptorSubtype; // 0x02 (FORMAT_TYPE)
   uint8_t  bFormatType;        // 0x01 (FORMAT_TYPE_I)
   uint8_t  bNrChannels;
   uint8_t  bSubframeSize;      // Container allocation in bytes: 1, 2, 3, or 4
   uint8_t  bBitResolution;     // Actual active bits: e.g., 16, 24
   uint8_t  bSamFreqType;       // 0 = continuous, n = number of discrete sample rates
};

// ========================================
// UAC2 Specific Descriptors (CS_INTERFACE: 0x24)
// ========================================

struct Uac2HeaderDescriptor {
   uint8_t  bLength;           // 0x09
   uint8_t  bDescriptorType;    // 0x24
   uint8_t  bDescriptorSubtype; // 0x01 (HEADER)
   uint16_t bcdADC;             // 0x0200
   uint8_t  bCategory;          // Primary category (e.g., Desktop Speaker, Headset)
   uint16_t wTotalLength;
   uint8_t  bmControls;
};

struct Uac2ClockSourceDescriptor {
   uint8_t  bLength;           // 0x08
   uint8_t  bDescriptorType;    // 0x24
   uint8_t  bDescriptorSubtype; // 0x0B (CLOCK_SOURCE)
   uint8_t  bClockID;
   uint8_t  bmAttributes;       // D1..0: 00=External, 01=Internal Fixed, 02=Internal Variable
   uint8_t  bmControls;         // D1..0: Clock Frequency Control (01=Read-Only, 03=Read/Write)
   uint8_t  bAssocTerminal;
   uint8_t  iClockSource;
};

struct Uac2InputTerminalDescriptor {
   uint8_t  bLength;           // 0x11
   uint8_t  bDescriptorType;    // 0x24
   uint8_t  bDescriptorSubtype; // 0x02 (INPUT_TERMINAL)
   uint8_t  bTerminalID;
   uint16_t wTerminalType;      // 0x0101: USB Streaming
   uint8_t  bAssocTerminal;
   uint8_t  bCSourceID;         // Associated Clock Entity ID
   uint8_t  bNrChannels;
   uint32_t bmChannelConfig;    // Spatial location bitmap
   uint8_t  iChannelNames;
   uint16_t bmControls;
```

```
    uint8_t  iTerminal;
};

struct Uac2FormatTypeIDescriptor {
    uint8_t  bLength;           // 0x06
    uint8_t  bDescriptorType;    // 0x24
    uint8_t  bDescriptorSubtype; // 0x02 (FORMAT_TYPE)
    uint8_t  bFormatType;        // 0x01 (FORMAT_TYPE_I)
    uint8_t  bSubslotSize;       // Physical slot allocation: 1, 2, 3, or 4 bytes
    uint8_t  bBitResolution;     // Valid audio resolution: e.g., 16, 24, 32 bits
};

#pragma pack(pop)

struct AudioStreamingEndpointConfiguration {
    uint8_t  interface_number{0};
    uint8_t  alternate_setting{0};
    uint8_t  protocol_version{0}; // 0x00 = UAC1, 0x20 = UAC2
    uint8_t  subslot_size{0};
    uint8_t  bit_resolution{0};
    uint8_t  channels{0};
    uint8_t  data_endpoint_address{0};
    uint8_t  feedback_endpoint_address{0};
    uint16_t max_packet_size{0};
    uint8_t  clock_source_id{0};
    bool     is_asynchronous{false};
    bool     is_implicit_feedback{false};
};

class AudioDescriptorParser {
public:
    static std::vector<AudioStreamingEndpointConfiguration> Parse(std::span<const uint8_t>
raw_config) {
        std::vector<AudioStreamingEndpointConfiguration> active_streams;
        size_t cursor = 0;

        std::optional<UsbInterfaceDescriptor> current_interface;
        std::optional<AudioStreamingEndpointConfiguration> active_stream_profile;
        uint8_t current_protocol = 0;
        uint8_t active_clock_source_id = 0;

        while (cursor + sizeof(UsbDescriptorHeader) <= raw_config.size()) {
            auto header = reinterpret_cast<const UsbDescriptorHeader*>(&raw_config[cursor]);
            if (header->bLength == 0 || (cursor + header->bLength) > raw_config.size()) {
                break; // Prevent infinite loops or over-reads on malformed bytes
            }

            std::span<const uint8_t> descriptor_bytes(&raw_config[cursor], header->bLength);
```

```
        switch (header->bDescriptorType) {
            case 0x04: { // USB_DT_INTERFACE
                if (descriptor_bytes.size() >= sizeof(UsbInterfaceDescriptor)) {
                    current_interface = *reinterpret_cast<const
UsbInterfaceDescriptor*>(descriptor_bytes.data());
                    if (current_interface->bInterfaceClass == 0x01) { // AUDIO Class
                        current_protocol = current_interface->bInterfaceProtocol;
                        if (current_interface->bInterfaceSubClass == 0x02) { // AUDIOSTREAMING
                            if (active_stream_profile.has_value()) {
                                active_streams.push_back(*active_stream_profile);
                            }
                            active_stream_profile = AudioStreamingEndpointConfiguration{
                                .interface_number = current_interface->bInterfaceNumber,
                                .alternate_setting = current_interface->bAlternateSetting,
                                .protocol_version = current_protocol,
                                .clock_source_id = active_clock_source_id
                            };
                        }
                    }
                }
                break;
            }

            case 0x24: { // CS_INTERFACE
                uint8_t subtype = descriptor_bytes[2];
                if (current_interface && current_interface->bInterfaceSubClass == 0x01) { //
AUDIOCONTROL
                    if (current_protocol == 0x20 && subtype == 0x0B) { // UAC2 CLOCK_SOURCE
                        if (descriptor_bytes.size() >= sizeof(Uac2ClockSourceDescriptor)) {
                            auto clk = reinterpret_cast<const
Uac2ClockSourceDescriptor*>(descriptor_bytes.data());
                            active_clock_source_id = clk->bClockID;
                        }
                    }
                } else if (current_interface && current_interface->bInterfaceSubClass == 0x02) { //
AUDIOSTREAMING
                    if (subtype == 0x02 && active_stream_profile) { // FORMAT_TYPE
                        if (current_protocol == 0x00 && descriptor_bytes.size() >=
sizeof(Uac1FormatTypeIDescriptor)) {
                            auto fmt = reinterpret_cast<const
Uac1FormatTypeIDescriptor*>(descriptor_bytes.data());
                            active_stream_profile->subslot_size = fmt->bSubframeSize;
                            active_stream_profile->bit_resolution = fmt->bBitResolution;
                            active_stream_profile->channels = fmt->bNrChannels;
                        } else if (current_protocol == 0x20 && descriptor_bytes.size() >=
sizeof(Uac2FormatTypeIDescriptor)) {
                            auto fmt = reinterpret_cast<const
```

```
Uac2FormatTypeIDescriptor*>(descriptor_bytes.data());
                        active_stream_profile->subslot_size = fmt->bSubslotSize;
                        active_stream_profile->bit_resolution = fmt->bBitResolution;
                    }
                }
            }
            break;
        }

        case 0x05: { // USB_DT_ENDPOINT
            if (descriptor_bytes.size() >= sizeof(UsbAudioEndpointDescriptor) &&
active_stream_profile) {
                auto ep = reinterpret_cast<const
UsbAudioEndpointDescriptor*>(descriptor_bytes.data());
                if ((ep->bmAttributes & 0x03) == 0x01) { // Isochronous
                    uint8_t sync_type = (ep->bmAttributes >> 2) & 0x03;
                    uint8_t usage_type = (ep->bmAttributes >> 4) & 0x03;

                    if (usage_type == 0x01) { // Explicit Feedback Endpoint
                        active_stream_profile->feedback_endpoint_address =
ep->bEndpointAddress;
                    } else if (usage_type == 0x00) { // Data Endpoint
                        active_stream_profile->data_endpoint_address = ep->bEndpointAddress;
                        active_stream_profile->max_packet_size = ep->wMaxPacketSize &
0x07FF;
                        active_stream_profile->is_asynchronous = (sync_type == 0x01);
                        if (sync_type == 0x01 && ep->bSynchAddress != 0) {
                            active_stream_profile->feedback_endpoint_address =
ep->bSynchAddress;
                        }
                    } else if (usage_type == 0x02) { // Implicit Feedback
                        active_stream_profile->is_implicit_feedback = true;
                        active_stream_profile->feedback_endpoint_address =
ep->bEndpointAddress;
                    }
                }
            }
            break;
        }

        default: break;
    }
    cursor += header->bLength;
}

if (active_stream_profile.has_value()) {
    active_streams.push_back(*active_stream_profile);
}
```

```
        return active_streams;
    }
};
```

Asynchronous Explicit and Implicit Feedback Loop Mechanics

Digital-to-analog converters operating in asynchronous USB mode decouple their master audio clock (MCLK) from the host's USB bus clock. The host delivers audio data synchronized to USB Start-of-Frame (SOF) markers emitted at 1\,\text{ms} intervals for Full-Speed USB, or microframes (\[span_24](start_span)[span_24](end_span)[span_39](start_span)[span_39](end_span)mu\text{SOF}) emitted every 125\,\mu\text{s} for High-Speed USB. Because the host's USB host controller oscillator and the DAC's local crystal oscillator exhibit frequency deviations of \pm 50\text{ to }2500\,\text{p[span_25](start_span)[span_25](end_span)[span_40](start_span)[span_40](end_span)pm}, sending an unvarying number of audio frames per transaction causes the DAC's physical First-In, First-Out (FIFO) buffer to drift toward underrun (audio dropouts) or overrun (crackle and dropped samples).

Under explicit feedback synchronization, the audio device measures its internal FIFO fullness and local crystal rate against the incoming USB frame markers and continuously streams the measured consumption rate back to the host via an Isochronous IN endpoint. Under implicit feedback synchronization, the device incorporates no discrete feedback endpoint; instead, the host determines the transmission rate by calculating the exact number of audio frames returned over a paired isochronous capture (microphone) stream per microframe.

Numerical Representations: 10.14 vs. 16.16 Fixed-Point Formats

The feedback payload encodes the exact instantaneous rate of audio frame consumption per USB bus transmission epoch. The physical encoding varies according to bus operating speed and specification revision.

USB 1.1 Full-Speed Feedback Format (10.14 Fixed-Point, 3 Bytes):
Byte 0[span_28](start_span)[span_28](end_span)[span_43](start_span)[span_43](end_span) [Bits 7..0] : Fractional Part LSB (f7..f0)
Byte 1 [Bits 15..8] : Fractional Part MSB ([span_29](start_span)[span_29](end_span)[span_44](start_span)[span_44](end_span)f13..f8) | Integer LSB (i1..i0)
Byte 2 [Bits 23..16] : Integer Part MSB (i9..i2)

USB 2.0 High-Speed Feedback Format (16.16 Fixed-Point, 4 Bytes):
Byte 0 [Bits 7..0] : Fractional Part LSB (f7..f0)
Byte 1 [Bits 15..8] : Fractional Part MSB (f15..f8)
Byte 2 [Bits 23..16] : Integer Part LSB (i7..i0)
Byte 3 [Bits 31..24] : Integer Part MSB (i15..i8)

For Full-Speed operation (1 ms interval frames), the device transmits 3 bytes formatted as unsigned 10.14 fixed-point values, representing the average number of samples consumed per $1\,\text{ms}$:
$$F_{10.14} = \text{Byte}_0 \mid (\text{Byte}_1 \ll 8) \mid (\text{Byte}_2 \ll 16)$$
For a standard $44.1\,\text{kHz}$ PCM stream, the nominal feedback value evaluates to:
$$F_{\text{nominal}} = 44.1 \implies 44.1 \times 2^{14} = 722534.4 \approx 0\text{x0B0666}$$
For High-Speed operation ($125\,\mu\text{s}$ microframes), the device transmits 4 bytes formatted as unsigned 16.16 fixed-point values, representing the average number of samples consumed per microframe (8000 microframes per second):
$$F_{16.16} = \text{Byte}_0 \mid (\text{Byte}_1 \ll 8) \mid (\text{Byte}_2 \ll 16) \mid (\text{Byte}_3 \ll 24)$$
For a standard $192.0\,\text{kHz}$ stream, the nominal feedback value evaluates to:
$$F_{\text{nominal}} = \frac{192000}{8000} = 24.0 \implies 24.0 \times 2^{16} = 1572864 = 0\text{x00180000}$$
Firmware implementations on certain audio bridges occasionally contain bugs where High-Speed endpoints report feedback values formatted as 10.14 packed into 3 bytes. The driver must detect the packet payload length returned by the kernel: 3-byte packets must be shifted left by 2 bits to normalize them into internal 16.16 phase space.

Jitter Attenuation and the Dynamic Packet-Sizing Engine

Raw feedback values extracted from USB Isochronous IN completion packets exhibit high-frequency jitter induced by bus schedule latency and PHY packet delivery variations. Directly feeding unfiltered feedback into packet sizing causes rapid oscillating frame allocations that violate USB audio specifications, which restrict microframe sample modulation to a narrow envelope (± 1 audio frame relative to nominal capacity). The driver must route incoming feedback samples through an infinite impulse response (IIR) single-pole low-pass filter, bounded by strict deviation thresholds.
The packet-sizing engine uses a 32-bit fixed-point fractional phase accumulator. During each microframe, the accumulator steps forward by the filtered feedback frequency value ($F \in Q16.16$). The integer sample allocation for microframe k equals the whole number generated by shifting the accumulator down by 16 bits; the lower 16 bits are retained to preserve fractional quantization history without phase error accumulation:

$$N_k = \lfloor \text{Accumulator}_k \gg 16 \rfloor, \quad \text{Accumulator}_{k+1} = (\text{Accumulator}_k \,\&\, 0\text{xFFFF}) + F_{\text{filtered}}$$

C++20 Usbfs Isochronous Execution Engine

Submitting and harvesting isochronous transfers through the Linux usbfs kernel subsystem requires managing the usbdevfs_urb and usbdevfs_iso_packet_desc structures. The driver initiates audio playback by allocating multiple multi-packet URBs (typically 4 to 8 URBs containing 8 isochronous microframes each, matching a $1\,\text{ms}$ servicing period), pre-filling their frame byte sizes, and queuing them to the kernel via ioctl(fd, USBDEVFS_SUBMITURB, &urb).
A dedicated real-time processing thread blocks on ioctl(fd, USBDEVFS_REAPURB, &reaped_urb). When an explicit feedback URB completes, its payload is parsed and filtered to update the phase accumulator. When an audio playback URB completes, the thread calculates the dynamic frame sizing for each microframe slot, copies audio data from an internal lock-free

ring buffer into the URB transfer buffer, and resubmits the URB to prevent endpoint starvation.
```
#pragma once

#include <linux/usbdevice_fs.h>
#include <sys/ioctl.h>
#include <cstdint>
#include <vector>
#include <thread>
#include <atomic>
#include <span>
#include <algorithm>
#include <memory>
#include <system_error>

class MicroframeFeedbackTracker {
public:
    enum class BusOperatingSpeed { FullSpeed, HighSpeed };

    MicroframeFeedbackTracker(uint32_t sample_rate, BusOperatingSpeed speed)
        : sample_rate_(sample_rate), speed_(speed) {
        if (speed_ == BusOperatingSpeed::HighSpeed) {
            nominal_rate_q16_ = (static_cast<uint64_t>(sample_rate_) << 16) / 8000;
        } else {
            nominal_rate_q16_ = (static_cast<uint64_t>(sample_rate_) << 16) / 1000;
        }
        filtered_rate_q16_ = nominal_rate_q16_;
    }

    void IngestFeedbackPayload(std::span<const uint8_t> packet) {
        uint64_t raw_feedback = 0;
        if (speed_ == BusOperatingSpeed::HighSpeed) {
            if (packet.size() >= 4) {
                raw_feedback = packet[0] | (packet[1] << 8) |
                               (packet[2] << 16) | (packet[3] << 24);
            } else if (packet.size() == 3) {
                // Non-standard bridge fallback: 10.14 transmitted over High-Speed
                raw_feedback = static_cast<uint64_t>(packet[0] | (packet[1] << 8) | (packet[2] << 16))
<< 2;
            }
        } else {
            if (packet.size() >= 3) {
                uint32_t raw_10_14 = packet[0] | (packet[1] << 8) | (packet[2] << 16);
                raw_feedback = static_cast<uint64_t>(raw_10_14) << 2; // Normalize to 16.16
            }
        }

        if (raw_feedback == 0) return;
```

```
        // Guard against irrational hardware feedback spikes (clamp to +/- 2500 ppm)
        uint64_t max_bound = nominal_rate_q16_ + (nominal_rate_q16_ * 2500 / 1000000);
        uint64_t min_bound = nominal_rate_q16_ - (nominal_rate_q16_ * 2500 / 1000000);

        if (raw_feedback >= min_bound && raw_feedback <= max_bound) {
            // First-order IIR filtering: Y[n] = (7 * Y[n-1] + X[n]) / 8
            filtered_rate_q16_ = ((filtered_rate_q16_ * 7) + raw_feedback) >> 3;
        }
    }

    uint32_t CalculateNextFrameAllocation() {
        phase_accumulator_ += filtered_rate_q16_;
        uint32_t allocated_frames = static_cast<uint32_t>(phase_accumulator_ >> 16);
        phase_accumulator_ &= 0xFFFF; // Maintain fractional phase residue
        return allocated_frames;
    }

private:
    uint32_t sample_rate_;
    BusOperatingSpeed speed_;
    uint64_t nominal_rate_q16_{0};
    uint64_t filtered_rate_q16_{0};
    uint64_t phase_accumulator_{0};
};

class ProductionAudioStreamer {
public:
    ProductionAudioStreamer(int usb_fd, uint8_t data_ep, uint8_t sync_ep,
                            uint32_t sample_rate, size_t channel_count, size_t subslot_size)
        : fd_(usb_fd), data_ep_(data_ep), sync_ep_(sync_ep),
          frame_bytes_(channel_count * subslot_size),
          tracker_(sample_rate, MicroframeFeedbackTracker::BusOperatingSpeed::HighSpeed) {}

    void Start(size_t packets_per_urb = 8, size_t urbs_in_flight = 4) {
        running_.store(true);
        stream_thread_ = std::jthread([this, packets_per_urb, urbs_in_flight]() {
            ProcessingWorker(packets_per_urb, urbs_in_flight);
        });
    }

    void Stop() {
        running_.store(false);
        if (stream_thread_.joinable()) {
            stream_thread_.join();
        }
    }

private:
```

```
struct InternalUrbContext {
    usbdevfs_urb urb;
    std::vector<usbdevfs_iso_packet_desc> descriptors;
    std::vector<uint8_t> payload_memory;
};

void ProcessingWorker(size_t packets_per_urb, size_t in_flight) {
    constexpr size_t kMaxPacketCapacity = 1024;
    std::vector<InternalUrbContext> playback_pool(in_flight);
    std::vector<InternalUrbContext> feedback_pool(in_flight);

    // Pre-allocate and submit explicit feedback URBs
    for (auto& fb_ctx : feedback_pool) {
        fb_ctx.descriptors.resize(packets_per_urb);
        fb_ctx.payload_memory.resize(packets_per_urb * 4);

        fb_ctx.urb.type = USBDEVFS_URB_TYPE_ISO;
        fb_ctx.urb.endpoint = sync_ep_;
        fb_ctx.urb.buffer = fb_ctx.payload_memory.data();
        fb_ctx.urb.buffer_length = static_cast<int>(fb_ctx.payload_memory.size());
        fb_ctx.urb.number_of_packets = static_cast<int>(packets_per_urb);
        fb_ctx.urb.usercontext = &fb_ctx;

        for (size_t i = 0; i < packets_per_urb; ++i) {
            fb_ctx.descriptors[i].length = 4;
        }
        ioctl(fd_, USBDEVFS_SUBMITURB, &fb_ctx.urb);
    }

    // Pre-allocate and submit audio playback URBs
    for (auto& pb_ctx : playback_pool) {
        pb_ctx.descriptors.resize(packets_per_urb);
        pb_ctx.payload_memory.resize(packets_per_urb * kMaxPacketCapacity);

        pb_ctx.urb.type = USBDEVFS_URB_TYPE_ISO;
        pb_ctx.urb.endpoint = data_ep_;
        pb_ctx.urb.buffer = pb_ctx.payload_memory.data();
        pb_ctx.urb.number_of_packets = static_cast<int>(packets_per_urb);
        pb_ctx.urb.usercontext = &pb_ctx;

        PopulatePlaybackPayload(pb_ctx, packets_per_urb);
        ioctl(fd_, USBDEVFS_SUBMITURB, &pb_ctx.urb);
    }

    while (running_.load(std::memory_order_relaxed)) {
        usbdevfs_urb* reaped_urb = nullptr;
        if (ioctl(fd_, USBDEVFS_REAPURB, &reaped_urb) < 0) {
            continue; // Handle EINTR or brief timeout
```

```
        }

        if (reaped_urb->endpoint == sync_ep_) {
            auto* fb_ctx = static_cast<InternalUrbContext*>(reaped_urb->usercontext);
            for (int i = 0; i < reaped_urb->number_of_packets; ++i) {
                if (fb_ctx->descriptors[i].status == 0 && fb_ctx->descriptors[i].actual_length >= 3) {
                    size_t offset = i * 4;
                    std::span<const uint8_t> payload(&fb_ctx->payload_memory[offset],
                                                     fb_ctx->descriptors[i].actual_length);
                    tracker_.IngestFeedbackPayload(payload);
                }
            }
            ioctl(fd_, USBDEVFS_SUBMITURB, reaped_urb);
        } else if (reaped_urb->endpoint == data_ep_) {
            auto* pb_ctx = static_cast<InternalUrbContext*>(reaped_urb->usercontext);
            PopulatePlaybackPayload(*pb_ctx, packets_per_urb);
            ioctl(fd_, USBDEVFS_SUBMITURB, reaped_urb);
        }
    }

    ioctl(fd_, USBDEVFS_DISCARDURB, nullptr);
}

void PopulatePlaybackPayload(InternalUrbContext& ctx, size_t packets_per_urb) {
    size_t write_offset = 0;
    for (size_t i = 0; i < packets_per_urb; ++i) {
        uint32_t frames_allocated = tracker_.CalculateNextFrameAllocation();
        size_t bytes_to_copy = frames_allocated * frame_bytes_;

        ctx.descriptors[i].length = static_cast<unsigned int>(bytes_to_copy);
        ctx.descriptors[i].actual_length = 0;
        ctx.descriptors[i].status = 0;

        SynthesizeAudioData(&ctx.payload_memory[write_offset], bytes_to_copy);
        write_offset += bytes_to_copy;
    }
    ctx.urb.buffer_length = static_cast<int>(write_offset);
}

void SynthesizeAudioData(uint8_t* destination, size_t bytes) {
    // Zero-fill buffer memory (silence) or drain lock-free ring buffer
    std::fill_n(destination, bytes, 0);
}

int fd_;
uint8_t data_ep_;
uint8_t sync_ep_;
size_t frame_bytes_;
```

```
    MicroframeFeedbackTracker tracker_;
    std::atomic<bool> running_{false};
    std::jthread stream_thread_;
};
```

Native DSD and DoP v1.1 USB Framing Architectures

Direct Stream Digital represents high-resolution audio as an oversampled 1-bit Pulse Density Modulation (PDM) signal sampled at direct multiples of 44.1\,\text{kHz}:
\text{DSD64} = 2.8224\,\text{MHz} = 64 \times 44.1\,\text{kHz}, \quad \text{DSD128} = 5.6448\,\text{MHz}, \quad \text{DSD256} = 11.2896\,\text{MHz}
Because standard operating systems and audio class stacks were historically designed for multi-bit Pulse Code Modulation (PCM), two discrete bitstreaming techniques exist for transmitting 1-bit streams across UAC2 endpoints: DSD over PCM (DoP v1.1) encapsulation and raw Native DSD bitstreaming (DSD_U32_BE).

DoP v1.1 Standard and Channel-Interleaved Bit-Packing

The DoP open standard encapsulates raw DSD without converting the 1-bit samples to multi-bit PCM, enabling bit-perfect transport across any standard UAC driver or operating system mixer that does not natively recognize DSD. DoP packs 16 serial bits of DSD audio into the lower two bytes of a standard 24-bit PCM container. The upper 8 bits (Bits 23..16) are reserved for a dynamic signature marker byte that alternates between 0x05 and 0xFA on consecutive audio frames. This rapid signature alternation allows downstream DAC logic to detect incoming DSD streams while protecting listeners: if a software volume slider or mixer alters the bitstream, the alternating marker pattern breaks, prompting the DAC to mute immediately rather than passing high-frequency DSD bits as full-scale noise.
DoP v1.1 Frame Structure in 24-bit / 32-bit Padded Containers:

```
Bit:   31          24 23              16 15              8 7               0
     +---------------+------------------+------------------+------------------+
     | Unused / 0x00 | DSD Marker Byte  | DSD Data MSB     | DSD Data LSB     |
     | Padding       | (0x05 / 0xFA)    | (b15..b8)        | (b7..b0)         |
     +---------------+------------------+------------------+------------------+
```

Because 16 bits of DSD are transmitted per stereo PCM frame, the host's underlying PCM sample rate evaluates to \frac{f_{\text{DSD}}}{16}. Streaming DSD64 requires configuring the UAC2 interface to a 176.4\,\text{kHz} PCM link; DSD128 requires a 352.8\,\text{kHz} link; and DSD256 requires a 705.6\,\text{kHz} link. The physical samples are channel-interleaved within each microframe slot:

```
#pragma once

#include <cstdint>
#include <span>
#include <algorithm>

class DopFramePacker {
```

```
public:
    DopFramePacker() : marker_(0x05) {}

    // Encapsulates raw stereo DSD bytes into 32-bit slot PCM structures (24-bit active)
    void PackStereoPayload(std::span<const uint8_t> dsd_ch0,
                           std::span<const uint8_t> dsd_ch1,
                           std::span<uint32_t> pcm_slots_out) {
        size_t frame_limit = std::min(dsd_ch0.size() / 2, dsd_ch1.size() / 2);
        size_t write_idx = 0;

        for (size_t i = 0; i < frame_limit; ++i) {
            // Ingest 16 bits of continuous DSD stream per channel (MSB transmitted first)
            uint16_t dsd_l = static_cast<uint16_t>((dsd_ch0[i * 2] << 8) | dsd_ch0[i * 2 + 1]);
            uint16_t dsd_r = static_cast<uint16_t>((dsd_ch1[i * 2] << 8) | dsd_ch1[i * 2 + 1]);

            // Construct 24-bit container in 32-bit slot: [0x00][Marker][DSD_MSB][DSD_LSB]
            uint32_t sample_l = (static_cast<uint32_t>(marker_) << 16) | dsd_l;
            uint32_t sample_r = (static_cast<uint32_t>(marker_) << 16) | dsd_r;

            pcm_slots_out[write_idx++] = sample_l;
            pcm_slots_out[write_idx++] = sample_r;

            // Invert the signature marker for the subsequent stereo frame
            marker_ = (marker_ == 0x05) ? 0xFA : 0x05;
        }
    }

private:
    uint8_t marker_;
};
```

Native DSD (DSD_U32_BE) Bitstreaming

While DoP v1.1 allows DSD streaming over unmodified PCM drivers, it introduces a 33\% bandwidth overhead due to the marker byte. Native DSD eliminates marker bytes entirely, delivering 32 bits of continuous DSD data per channel in each 32-bit audio slot. This doubling of transmission efficiency allows High-Speed USB 2.0 endpoints to reach higher bitrates such as DSD512 and DSD1024.

Under ALSA and hardware bridge specifications, native streaming is standardized around the DSD_U32_BE format. The stream consists of 32-bit words packed in **Big-Endian** order: the chronologically earliest 1-bit DSD sample resides in bit 31 (MSB) of the highest byte, and the 32nd sample terminates in bit 0 of the lowest byte. The stereo stream alternates continuously between 32-bit channel blocks: \text{Left}_0, \text{Right}_0, \text{Left}_1, \[span_99](start_span)[span_99](end_span)[span_103](start_span)[span_103](end_span)text{Right}_1, \dots.

```
class NativeDsdPacker {
public:
```

```
    static void PackStereoNativeU32BE(std::span<const uint8_t> dsd_ch0,
                                      std::span<const uint8_t> dsd_ch1,
                                      std::span<uint32_t> output_stream) {
        size_t word_count = std::min(dsd_ch0.size() / 4, dsd_ch1.size() / 4);
        size_t out_idx = 0;

        for (size_t i = 0; i < word_count; ++i) {
            size_t in_offset = i * 4;

            // Assemble 32-bit words preserving temporal big-endian bit alignment
            uint32_t word_l = (static_cast<uint32_t>(dsd_ch0[in_offset + 0]) << 24) |
                              (static_cast<uint32_t>(dsd_ch0[in_offset + 1]) << 16) |
                              (static_cast<uint32_t>(dsd_ch0[in_offset + 2]) << 8)  |
                              (static_cast<uint32_t>(dsd_ch0[in_offset + 3]));

            uint32_t word_r = (static_cast<uint32_t>(dsd_ch1[in_offset + 0]) << 24) |
                              (static_cast<uint32_t>(dsd_ch1[in_offset + 1]) << 16) |
                              (static_cast<uint32_t>(dsd_ch1[in_offset + 2]) << 8)  |
                              (static_cast<uint32_t>(dsd_ch1[in_offset + 3]));

            output_stream[out_idx++] = word_l;
            output_stream[out_idx++] = word_r;
        }
    }
};
```

Vendor-Specific Alternate Setting Activation Quirks

Because the UAC2 specification does not define an explicit format tag for raw 1-bit DSD bitstreams, hardware bridge manufacturers handle native DSD through proprietary alternate settings or control requests. The user-space driver must detect the target hardware bridge and execute vendor-specific initialization commands before streaming audio.

Controller Architecture	Vendor ID (VID)	Detection Pattern	Native Initialization and Alternate Setting Activation Sequence
XMOS xCORE (200/XU316) / Thesycon	0x20B1, 0x152A	Alternate Setting 2 or 3 exposes RAW_DATA (0x0001).	Activate target Alternate Setting via USBDEVFS_SETINTERFACE. Submit 64 ms of zeroed DSD buffers before sending valid audio to initialize internal decimation filters.
Comtrue (CT7601)	0x2FC6	Dual operational streaming alternate	Switch to Alternate Setting 2. Issue a

Controller Architecture	Vendor ID (VID)	Detection Pattern	Native Initialization and Alternate Setting Activation Sequence
		settings (PCM vs DSD).	class-specific control request to Clock Entity 0x01 setting the clock frequency to the target DSD rate.
Savitech (SA9227/9312)	0x262A	Shared PCM/DSD alternate interface descriptors.	Issue a vendor-specific control transfer on EP0: bmRequestType = 0x41, bRequest = 0x01, wValue = 0x0001, wIndex = InterfaceNumber to engage native DSD mode.
Cirrus Logic / Technics (CS43131 Bridges)	0x3302, 0x04DA	Composite streaming interfaces; UAC2 protocol standard.	Switch to Alternate Setting 3. Issue vendor control write bmRequestType = 0x41, bRequest = 0x00, wValue = 0x0001, wIndex = 0x0001 to unmute the internal DSD processing pipeline.

Type-C DSP IEM USB HID Parametric Equalizer (PEQ) Protocols

High-performance Type-C digital earphones and micro-DAC dongles integrate ultra-low-power DSP engines directly into their USB interface controllers. These devices enumerate as composite USB audio peripherals: the audio stream routes to a standard UAC interface, while parameter read/write commands route to a collocated USB Human Interface Device (HID) interface (bInterfaceClass = 0x03). This architecture allows the host driver to dynamically reprogram onboard biquad filter banks, gains, limiter thresholds, and DAC reconstruction filters over USB.

Standard USB HID Control Request Mechanism

Configuring PEQ parameters over usbfs uses USB Control transfers targeting the default control pipe (Endpoint 0) via the USBDEVFS_CONTROL ioctl:

- **SET_REPORT:** Host-to-device parameter dispatch: bmRequestType = 0x21, bRequest = 0x09. The 16-bit wValue field encodes the Report Type in its upper byte (0x03 for Feature Reports, 0x02 for Output Reports) and the Report ID in its lower byte. The wIndex field addresses the zero-based HID interface number.

- **GET_REPORT:** Device-to-host register read-back: bmRequestType = 0xA1, bRequest = 0x01, with matching wValue and wIndex structures.

```
#include <linux/usbdevice_fs.h>
#include <sys/ioctl.h>
#include <cstdint>
#include <span>
#include <system_error>

class UsbHidControlTransport {
public:
    UsbHidControlTransport(int fd, uint8_t hid_interface)
        : fd_(fd), interface_(hid_interface) {}

    void WriteFeatureReport(uint8_t report_id, std::span<const uint8_t> payload) {
        usbdevfs_ctrltransfer transfer{};
        transfer.bRequestType = 0x21; // Host-to-Device | Class-Specific | Interface
        transfer.bRequest = 0x09;     // HID SET_REPORT
        transfer.wValue = static_cast<uint16_t>((0x03 << 8) | report_id); // 0x03 = Feature Report
        transfer.wIndex = interface_;
        transfer.wLength = static_cast<uint16_t>(payload.size());
        transfer.data = const_cast<uint8_t*>(payload.data());
        transfer.timeout = 1000;

        if (ioctl(fd_, USBDEVFS_CONTROL, &transfer) < 0) {
            throw std::system_error(errno, std::generic_category(), "HID SET_REPORT failure");
        }
    }

    void ReadFeatureReport(uint8_t report_id, std::span<uint8_t> destination) {
        usbdevfs_ctrltransfer transfer{};
        transfer.bRequestType = 0xA1; // Device-to-Host | Class-Specific | Interface
        transfer.bRequest = 0x01;     // HID GET_REPORT
        transfer.wValue = static_cast<uint16_t>((0x03 << 8) | report_id);
        transfer.wIndex = interface_;
        transfer.wLength = static_cast<uint16_t>(destination.size());
        transfer.data = destination.data();
        transfer.timeout = 1000;

        if (ioctl(fd_, USBDEVFS_CONTROL, &transfer) < 0) {
            throw std::system_error(errno, std::generic_category(), "HID GET_REPORT failure");
        }
    }

private:
    int fd_;
    uint8_t interface_;
};
```

Analysis of Production DSP Protocols

1. KTMicro DSP Architecture (KT0210, KT0211L, KT02H20, KT0231H)

KTMicro controllers are widely used in low-cost, high-performance DSP products, including the FiiO JA11, Simgot EW100 DSP, and JCally JM12. Parameters are modified through direct read/write operations against an internal register map via 8-byte or 64-byte Feature Reports targeting Report ID 0x00 or 0x08.
Commands use an initial opcode byte (0x01 Read, 0x02 Write, 0x03 Commit to Non-Volatile Memory) followed by the base register address, data length, and 32-bit payload words. Hardware biquad filters are mapped across registers 0x34 through 0x4D on the KT0210/KT0211L (providing 5 parametric bands), and up to register 0x5B on the KT0231H (providing 6 DAC and 6 ADC bands). Each filter band is controlled by two consecutive 32-bit registers:
KTMicro [span_164](start_span)[span_164](end_span)PEQ Register Bit Allocations:
Register 0: [Frequency in Hz (Bits 31..16)] [Gain in 0.1 dB (Bits 15..0, Signed Two's Complement)]
Register 1: [Filter Type (Bits 31..24)] [Quality Factor Q * 1000 (Bits 23..0, Unsigned Integer)]

Filter types are defined as: 0x00 for Peaking, 0x01 for Low Shelf, and 0x02 for High Shelf. Writing 0x00[span_165](start_span)[span_165](end_span)C80050 to Register 0x35 and 0x000302BC to Register 0x36 configures a Low-Shelf filter centered at 200\,\text{Hz} with +8.0\,\text{dB} of gain and a Q factor of 0.700.

2. WalkPlay / Sentry Ecosystem

Common in devices built around Comtrue bridges and modern audiophile cables, including the CrinEar Protocol Micro/Max, Celest Wyvern Abyss, and Moondrop DSP interconnects. The WalkPlay protocol communicates through 64-byte HID reports with a fixed frame header:
WalkPlay 64-Byte PEQ Frame Layout:
Byte 0 : Magic Synchronization Marker (0x5A)
Byte 1 : Command Opcode (0x20 Write Band, 0x21 Read Band, 0x24 Set Preamp, 0x2F Commit)
Byte 2 : Target Filter Band Index (0x00 to 0x09 for 10-band support)
Byte 3 : Filter Characteristic (0=Bypass, 1=Peak, 2=Low-Shelf, 3=High-Shelf, 4=LPF, 5=HPF)
Bytes 4..5 : Center / Corner Frequency in Hertz (16-bit Big-Endian Unsigned Integer)
Bytes 6..7 : Gain in Centibels (0.01 dB steps; 16-bit Big-Endian Signed Two's Complement)
Bytes 8..9 : Quality Factor Q * 100 (16-bit Big-Endian Unsigned Integer; e.g., 71 for Q=0.71)
Bytes 10..63 : Trailing Zero-Padding (Reserved)

3. Moondrop / Tanchjim FreeDSP Direct Biquad Engine

High-end audiophile cables such as the Moondrop FreeDSP, MAY, DUSK-SP, and Tanchjim

Tanya DSP offload coefficient calculation entirely to the host application. Rather than configuring high-level acoustic parameters (f_0, \text{Gain}, Q), the driver computes the five second-order IIR biquad transfer coefficients in software, normalizes them, converts them to signed fixed-point format, and writes them directly into the hardware processing engine.
The discrete Direct Form I transfer function is defined as:
H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}} = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{a_0 + a_1 z^{-1} + a_2 z^{-2}} \quad (\text{where } a_0 = 1.0)
Coefficients are mapped to 32-bit signed fixed-point values using Q2.30 or Q5.27 encoding. The driver transmits the five coefficients via Feature Report 0x08 using the sequence \{b_0, b_1, b_2, -a_1, -a_2\}, where the feedback coefficients are sign-inverted to match hardware multiply-accumulate topologies.
DAC reconstruction filter profiles are selected using an 8-byte Feature Report targeting Report ID 0x07:
Byte 0 : Report ID (0x07)
Byte 1 : Reconstruction Filter Profile:
    0x00 = Fast Roll-off, Linear Phase
    0x01 = Slow Roll-off, Linear Phase
    0x0[span_158](start_span)[span_158](end_span)2 = Fast Roll-off, Minimum Phase
    0x03 = Slow Roll-off, Minimum Phase
    0x04 = Non-Oversampling (NOS) Direct Filter Bypass

4. Qudelix Hardware Control Protocol

The Qudelix series exposes a proprietary binary packet protocol across HID Interface 2 (using Report ID 0x02). Filter updates are transmitted as 64-byte payload frames using IEEE 754 32-bit single-precision floating-point values packed in Little-Endian byte order:
Byte 0 : Report ID (0x02)
Byte 1 : Subsystem Opcode (0xC1 = Equalizer Bank, 0xC2 = DAC Profile Configuration)
Byte 2 : Equalizer State (0x00 = Bypass, 0x01 = Acti[span_159](start_span)[span_159](end_span)ve)
Byte 3 : Band Index (0x00 through 0x09)
Bytes 4..7 : Filter Center Frequency (IEEE 754 32-bit float, Little-Endian)
Bytes 8..11 : Band Gain in dB (IEEE 754 32-bit float, Little-Endian)
Bytes 12..15 : Quality Factor Q (IEEE 754 32-bit float, Little-Endian)
Bytes 16..19 : Filter Topology Type (0 = Peaking, 1 = Low Shelf, 2 = High Shelf)
Bytes 20..23 : Global Pre-Equalization Gain in dB (IEEE 754 32-bit float)
Bytes 24..63 : Zero-Padding

Comparative Analysis of Production DSP Architectures

The operational boundaries, filter calculation domains, and hardware register access models across these DSP bridges are summarized below:

Target DSP Hardware	HID Interface	Report ID & Addressing	Parameter Domain	Maximum Sampling Target	DAC Hardware Filter Control
KTMicro KT0210/0211L	HID Class (0x03)	Report 0x00/0x08, Regs 0x34..0x4D	Hardware-Calculated (f_c, \text{Gain}, Q)	96\,\text{kHz} / 24-bit	Mute register only; fixed minimum-phase filter.
KTMicro KT0231H	HID Class (0x03)	Report 0x00/0x08, Regs 0x34..0x5B	Hardware-Calculated (6 DAC + 6 ADC Bands)	384\,\text{kHz} / 32-bit, DSD128	Register programmable roll-off modes.
WalkPlay (Comtrue/OEM)	HID Class (0x03)	Report 0x00, 64-Byte Payload	Microcontroller-Calculated (f_c, \te[span_168](start_span)[span_168](end_span)xt{Gain}, Q)	768\,\text{kHz} / 32-bit, DSD256	Software selectable via Opcode 0x3A.
Moondrop / Tanchjim FreeDSP	HID Class (0x03)	Report 0x08 (Biquad), 0x07 (DAC Config)	Host Software-Synthesized (Q2.30 Biquads)	192\,\text{kHz} / 24-bit	5 hardware reconstruction profiles via Reg 0x07.
Qudelix-5K	HID Class (0x03)	Report 0x02, Sub-commands 0xC1/0xC2	Direct IEEE 754 Floating-Point Parameters	96\,\text{kHz} / 24-bit (Dual ES9219)	8 custom roll-off filters mapped over Opcode 0xC2.

C++20 Biquad Synthesizer and HID Dispatch Engine

The C++20 implementation below calculates exact second-order IIR biquad filter coefficients for Peaking, Low-Shelf, and High-Shelf equalization filters using standard digital audio cookbook formulas, normalizes them by a_0, converts them to signed Q2.30 fixed-point values, and serializes the parameter payloads into both WalkPlay and FreeDSP transport formats:

```
#pragma once

#include <cmath>
#include <cstdint>
#include <array>
#include <span>
#include <numbers>
#include <algorithm>

class ProductionDspSynthesizer {
public:
    enum class FilterTopology { Peaking, LowShelf, HighShelf };

    struct EqualizerBandSpecification {
        FilterTopology topology;
        double center_frequency_hz;
```

```
        double gain_db;
        double quality_factor_q;
    };

    struct NormalizedBiquadQ30 {
        int32_t b0;
        int32_t b1;
        int32_t b2;
        int32_t a1; // Inverted sign for standard Direct Form I accumulation
        int32_t a2; // Inverted sign
    };

    static NormalizedBiquadQ30 SynthesizeBiquad(const EqualizerBandSpecification& spec,
double sample_rate) {
        double A = std::pow(10.0, spec.gain_db / 40.0);
        double w0 = 2.0 * std::numbers::pi * (spec.center_frequency_hz / sample_rate);
        double cos_w0 = std::cos(w0);
        double sin_w0 = std::sin(w0);
        double alpha = sin_w0 / (2.0 * spec.quality_factor_q);

        double b0 = 0.0, b1 = 0.0, b2 = 0.0;
        double a0 = 1.0, a1 = 0.0, a2 = 0.0;

        switch (spec.topology) {
            case FilterTopology::Peaking: {
                b0 = 1.0 + (alpha * A);
                b1 = -2.0 * cos_w0;
                b2 = 1.0 - (alpha * A);
                a0 = 1.0 + (alpha / A);
                a1 = -2.0 * cos_w0;
                a2 = 1.0 - (alpha / A);
                break;
            }
            case FilterTopology::LowShelf: {
                double sqrt_A = std::sqrt(A);
                double alpha_shelf = sin_w0 / 2.0 * std::sqrt((A + 1.0 / A) * (1.0 / spec.quality_factor_q
- 1.0) + 2.0);

                b0 = A * ((A + 1.0) - (A - 1.0) * cos_w0 + 2.0 * sqrt_A * alpha_shelf);
                b1 = 2.0 * A * ((A - 1.0) - (A + 1.0) * cos_w0);
                b2 = A * ((A + 1.0) - (A - 1.0) * cos_w0 - 2.0 * sqrt_A * alpha_shelf);
                a0 = (A + 1.0) + (A - 1.0) * cos_w0 + 2.0 * sqrt_A * alpha_shelf;
                a1 = -2.0 * ((A - 1.0) + (A + 1.0) * cos_w0);
                a2 = (A + 1.0) + (A - 1.0) * cos_w0 - 2.0 * sqrt_A * alpha_shelf;
                break;
            }
            case FilterTopology::HighShelf: {
                double sqrt_A = std::sqrt(A);
```

```
            double alpha_shelf = sin_w0 / 2.0 * std::sqrt((A + 1.0 / A) * (1.0 / spec.quality_factor_q
- 1.0) + 2.0);

            b0 = A * ((A + 1.0) + (A - 1.0) * cos_w0 + 2.0 * sqrt_A * alpha_shelf);
            b1 = -2.0 * A * ((A - 1.0) + (A + 1.0) * cos_w0);
            b2 = A * ((A + 1.0) + (A - 1.0) * cos_w0 - 2.0 * sqrt_A * alpha_shelf);
            a0 = (A + 1.0) - (A - 1.0) * cos_w0 + 2.0 * sqrt_A * alpha_shelf;
            a1 = 2.0 * ((A - 1.0) - (A + 1.0) * cos_w0);
            a2 = (A + 1.0) - (A - 1.0) * cos_w0 - 2.0 * sqrt_A * alpha_shelf;
            break;
        }
    }

    // Normalize coefficients by a0
    double norm_b0 = b0 / a0;
    double norm_b1 = b1 / a0;
    double norm_b2 = b2 / a0;
    double norm_a1 = a1 / a0;
    double norm_a2 = a2 / a0;

    // Scale to signed Q2.30 fixed-point representation (Scale = 2^30)
    constexpr double kQ30Scale = 1073741824.0;

    return NormalizedBiquadQ30{
        .b0 = static_cast<int32_t>(std::clamp(norm_b0 * kQ30Scale, -2147483648.0,
2147483647.0)),
        .b1 = static_cast<int32_t>(std::clamp(norm_b1 * kQ30Scale, -2147483648.0,
2147483647.0)),
        .b2 = static_cast<int32_t>(std::clamp(norm_b2 * kQ30Scale, -2147483648.0,
2147483647.0)),
        .a1 = static_cast<int32_t>(std::clamp(-norm_a1 * kQ30Scale, -2147483648.0,
2147483647.0)),
        .a2 = static_cast<int32_t>(std::clamp(-norm_a2 * kQ30Scale, -2147483648.0,
2147483647.0))
    };
  }

  static std::array<uint8_t, 64> SerializeWalkPlayFrame(uint8_t band_index,
                                        const EqualizerBandSpecification& spec) {
    std::array<uint8_t, 64> payload{};
    payload[0] = 0x5A; // Magic marker
    payload[1] = 0x20; // Opcode: Write Filter
    payload[2] = band_index;

    switch (spec.topology) {
        case FilterTopology::Peaking:   payload[3] = 0x01; break;
        case FilterTopology::LowShelf:  payload[3] = 0x02; break;
        case FilterTopology::HighShelf: payload[3] = 0x03; break;
```

```
        }

        uint16_t freq = static_cast<uint16_t>(spec.center_frequency_hz);
        payload[4] = static_cast<uint8_t>((freq >> 8) & 0xFF);
        payload[5] = static_cast<uint8_t>(freq & 0xFF);

        int16_t gain_cdb = static_cast<int16_t>(spec.gain_db * 100.0);
        payload[6] = static_cast<uint8_t>((gain_cdb >> 8) & 0xFF);
        payload[7] = static_cast<uint8_t>(gain_cdb & 0xFF);

        uint16_t q_scaled = static_cast<uint16_t>(spec.quality_factor_q * 100.0);
        payload[8] = static_cast<uint8_t>((q_scaled >> 8) & 0xFF);
        payload[9] = static_cast<uint8_t>(q_scaled & 0xFF);

        return payload;
    }

    static std::array<uint8_t, 64> SerializeFreeDspBiquadFrame(uint8_t band_index,
                                                const NormalizedBiquadQ30& coeffs) {
        std::array<uint8_t, 64> report{};
        report[0] = 0x08;       // Report ID
        report[1] = 0x57;       // Opcode: Set Filter Bank Biquad
        report[2] = band_index; // Band index (0x00 to 0x09)
        report[3] = 20;         // Payload byte length (5 * 4-byte integers)

        auto write_word_le = [&report](size_t offset, int32_t value) {
            report[offset + 0] = static_cast<uint8_t>(value & 0xFF);
            report[offset + 1] = static_cast<uint8_t>((value >> 8) & 0xFF);
            report[offset + 2] = static_cast<uint8_t>((value >> 16) & 0xFF);
            report[offset + 3] = static_cast<uint8_t>((value >> 24) & 0xFF);
        };

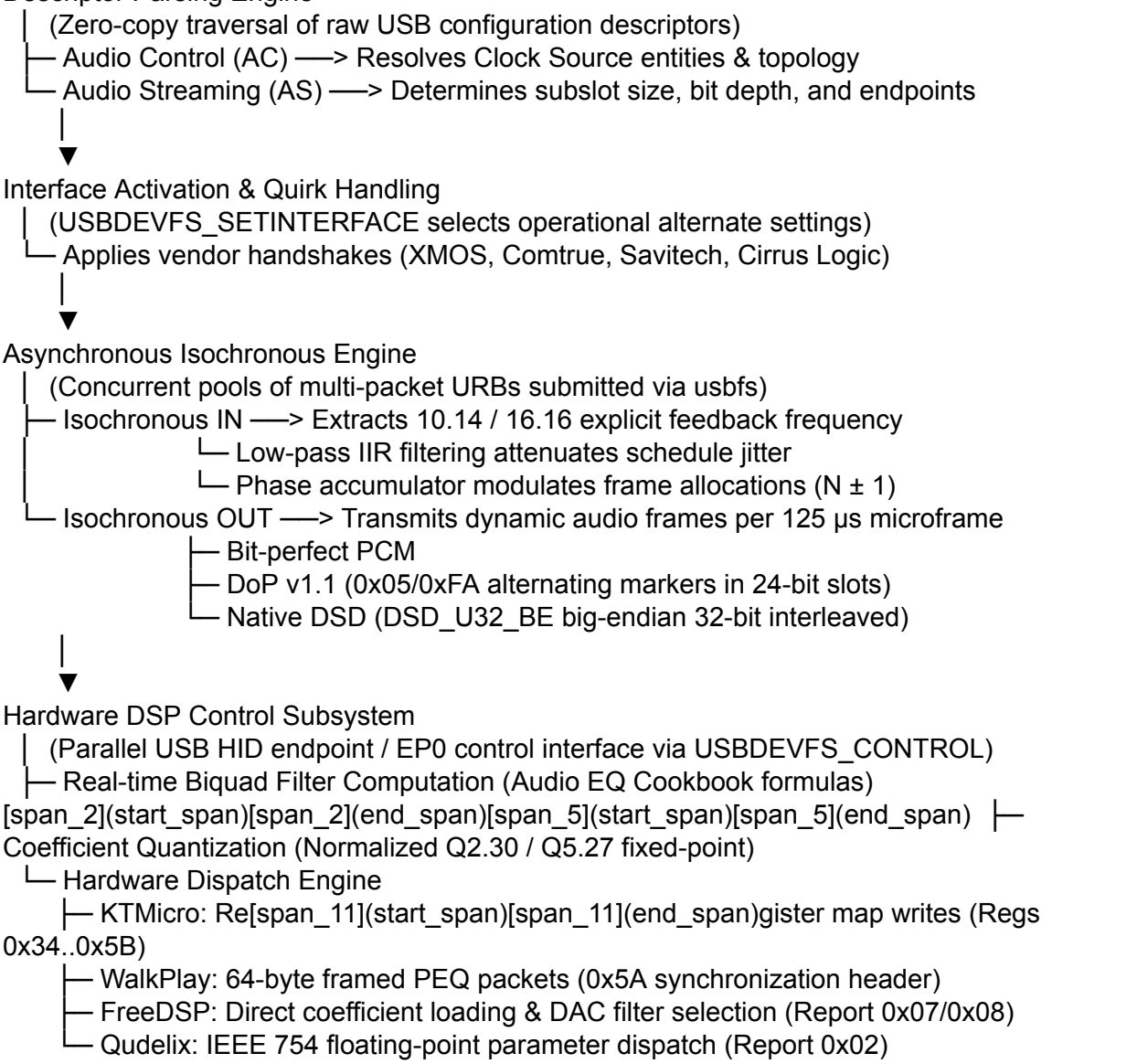
        write_word_le(4,  coeffs.b0);
        write_word_le(8,  coeffs.b1);
        write_word_le(12, coeffs.b2);
        write_word_le(16, coeffs.a1);
        write_word_le(20, coeffs.a2);

        return report;
    }
};
```

Driver Integration and Subsystem Execution

Building a production-ready C++20 user-space USB audio driver on Android requires orchestrating these low-level components into a cohesive, deterministic pipeline. The driver

begins by retrieving the raw device handle via UsbDeviceConnection.getFileDescriptor(), taking exclusive control of the USB interfaces.



The system initializes by using AudioDescriptorParser to resolve the audio control topology, locate clock sources, identify operational alternate settings, and map both data and feedback endpoints. If high-resolution native DSD playback is requested, the driver checks the device's vendor and product IDs against known hardware profiles, issues any required vendor-specific control transfers, and activates the matching alternate setting.

Once streaming starts, the playback pipeline runs on a dedicated, high-priority real-time thread. The thread submits concurrent pools of multi-packet URBs to the kernel using USBDEVFS_SUBMITURB, and continuously reaps completions with USBDEVFS_REAPURB. The MicroframeFeedbackTracker continuously ingests explicit or implicit feedback packets, applies single-pole low-pass filtering to attenuate microframe timing jitter, and advances a fractional phase accumulator to compute the exact frame count (N or N \pm 1) for each microframe. Audio frames are formatted into the playback URBs—packing standard PCM, DoP

v1.1 with alternating markers, or native DSD_U32_BE bitstreams—ensuring sample-accurate delivery that prevents DAC buffer overruns and underruns.
In parallel with audio streaming, the driver operates the USB HID control subsystem to configure hardware-level DSP. Acoustic adjustments—such as target frequencies, gains, and Q factors—are processed by ProductionDspSynthesizer, which computes second-order IIR biquad transfer functions and maps them into the representation required by the detected bridge. The driver then dispatches these parameters across the HID interface using USBDEVFS_CONTROL: updating register banks on KTMicro chips, sending framed packets on WalkPlay bridges, loading normalized Q2.30 coefficient taps on FreeDSP cables, or transmitting floating-point vectors to Qudelix controllers.
By unifying descriptor discovery, microframe-synchronized asynchronous transfer loops, low-overhead bitstreaming, and vendor-specific HID control within a single C++20 engine, this architecture provides a complete foundation for high-performance, bit-perfect audiophile playback and hardware DSP processing on Android.

Works cited

1. linux/drivers/usb/core/devio.c at master - GitHub, https://github.com/torvalds/linux/blob/master/drivers/usb/core/devio.c 2. how to get usb's urb info in linux - Stack Overflow, https://stackoverflow.com/questions/11574418/how-to-get-usbs-urb-info-in-linux 3. The Linux-USB Host Side API, https://www.kernel.org/doc/html/v4.14/driver-api/usb/usb.html 4. UAC Feedback端点- USB中文网, https://www.usbzh.com/article/detail-274.html 5. Inside an Embedded Linux Audio Product: USB DSD Quirks, Kernel, https://www.grinaldssolutions.com/blog/embedded-linux-audio-product-development.html 6. I built a Web App to unlock the hardware PEQ on the Savitech, https://www.reddit.com/r/iems/comments/1ty9xy9/i_built_a_web_app_to_unlock_the_hardware_peq_on/ 7. rpi-5.15.y drivers/usb/gadget uac2 not working in windows - GitHub, https://github.com/raspberrypi/linux/issues/4587 8. UAC音频同步实现 - USB中文网, https://www.usbzh.com/article/detail-763.html 9. Audio doesn't work properly on STM32U5 · Issue #92 - GitHub, https://github.com/eclipse-threadx/usbx/issues/92 10. STM32 USB to I2S multi channel - log - ask for help | diyAudio, https://www.diyaudio.com/community/threads/stm32-usb-to-i2s-multi-channel-log-ask-for-help.409771/ 11. USB Audio Class 2.0 - How to support multiple bit rates/sample rates, https://stackoverflow.com/questions/41702932/usb-audio-class-2-0-how-to-support-multiple-bit-rates-sample-rates 12. CVE-2026-98265 Common Vulnerabilities and Exposures | SUSE, https://www.suse.com/ko-kr/security/cve/CVE-2026-98265.html 13. Re: [alsa-devel] [PATCH 1/1] ALSA: usb: initial USB Audio Device, https://mailman.alsa-project.org/hyperkitty/list/alsa-devel@alsa-project.org/message/WVWP3UGT6W3RFTAHKGO4BLVKU427N63G/ 14. USB Audio Sync Endpoint Implementierung - Mikrocontroller.net, https://www.mikrocontroller.net/topic/470707 15. USB音频声卡的时钟同步方式同步、自适应、异步的理解和优缺点, https://bbs.erji.net/forum.php?mod=viewthread&tid=2076663 16. [PATCH 5.4 48/65] Revert "ALSA: usb-audio: Improve frames size, http://www.mail-archive.com/linux-kernel@vger.kernel.org/msg2224998.html 17. linux-dev - Linux kernel development work - see feature branches, https://git.zx2c4.com/linux-dev/diff/sound/usb/endpoint.c?id=ff0700f03609b9f0defacd4ce96d9519d721e0a2 18. DSD native 與DoP 差在哪？Linux MPD 設定與bit-perfect 驗證,

https://lingyinaudio.com/differences-between-dsd-native-and-dop/ 19. DSD Direct (again) - Help - Volumio Community, https://community.volumio.com/t/dsd-direct-again/9147 20. ALSA: pcm: Add big-endian DSD sample formats and fix XMOS DSD, https://gitgud.io/bkeys/kernel-reform/-/commit/d42472ecffd7c42086c6e5b1335c99a3adf58a09 21. 不少人花錢買的DSD，可能根本沒正確送到你的DAC。而且設定錯誤, https://www.facebook.com/groups/843368775764250/posts/27605578405783257/ 22. Доработка MPD (C++, Linux) - Конференция iXBT.com, https://forum.ixbt.com/topic.cgi?id=94:103 23. [海外交流-訪馬來西亞Centre Circle Audio] 經由【Creative AV】林, https://www.facebook.com/groups/843368775764250/posts/8823653384402376/ 24. Technics SL-G700 and SL-G700m2 - Roon Ready?, https://community.roonlabs.com/t/technics-sl-g700-and-sl-g700m2-roon-ready/229288?page=2 25. Which Software for RPi4 to stream multichannel DSD?, https://www.audiosciencereview.com/forum/index.php?threads/which-software-for-rpi4-to-stream-multichannel-dsd.19340/ 26. DSD hardware quirks and usb vendor id hardcoding - Alsa-devel, https://mailman.alsa-project.org/hyperkitty/list/alsa-devel@alsa-project.org/thread/VQMGFFWNRVKWBL4NCGBIQ5DLQD4SJK2U/ 27. linux/sound/usb/quirks.c at master · torvalds/linux - GitHub, https://github.com/torvalds/linux/blob/master/sound/usb/quirks.c 28. FiiO has released the USBDAC JA11, with an initial price of only $9, https://www.audiosciencereview.com/forum/index.php?threads/fiio-has-released-the-usbdac-ja11-with-an-initial-price-of-only-9-69rmb-and-support-for-peq-dsd128-pcm384.55160/page-29 29. Hub_Moon/moondrop_hub_reverse_engineering.md at main - GitHub, https://github.com/MiyukiVigil/Hub_Moon/blob/main/moondrop_hub_reverse_engineering.md 30. Qudelix-5k USB udev rules - GitHub Gist, https://gist.github.com/hmtheboy154/21c0a25ff025667981a35b6656f7da69 31. Supported devices - Contour, https://contoureq.app/devices/ 32. Contour guide, https://contoureq.app/guide/ 33. contour/THIRD_PARTY_NOTICES.md at main · Chronosauros/contour, https://github.com/Chronosauros/contour/blob/main/THIRD_PARTY_NOTICES.md 34. EffectiveEquivalent/FreeDSP-Studio · GitHub - GitHub, https://github.com/EffectiveEquivalent/FreeDSP-Studio 35. devicePEQ - jeromeof - GitHub, https://github.com/jeromeof/devicePEQ 36. I reverse-engineered my DUSK-SP cable and built my own EQ app, https://www.reddit.com/r/inearfidelity/comments/1vmar1z/i_reverseengineered_my_dusksp_cable_and_built_my/ 37. freeDSP V2.0 (ADAU1452) developement thread | Page 5 - diyAudio, https://www.diyaudio.com/community/threads/freedsp-v2-0-adau1452-developement-thread.268660/page-5 38. Changelog in Linux kernel 6.16.10 - Opennet.ru, https://www.opennet.ru/kernel/6.16.10.html